



INTERNSHIP PROJECT PRESENTATION

ShopFusion

“Where Shopping Meets Innovation” — A Modern Full-Stack E-Commerce Architecture

An enterprise-grade, high-performance web application engineered to solve disjointed customer workflows through unified catalog discovery, secure authentication, atomic transactions, and automated order fulfillment.

STUDENT NAME Vikas Yadav	ENROLLMENT NO CS-2341804	PROGRAM & SEMESTER B.Tech (CSE) • Semester 7th	ORGANIZATION ShopFusion Tech Labs	PROJECT DOMAIN Full-Stack Web Engineering
-----------------------------	-----------------------------	---	--------------------------------------	--



The Fragmentation Problem

Conventional e-commerce platforms suffer from fragmented operational layers: unoptimized server responses, unstable client-side cart states, vulnerable session management, and complex payment handoffs.

- Disjointed user state between guest and authenticated sessions.
- High checkout abandonment due to multi-step redirect latency.
- Inventory race conditions during concurrent user checkouts.



The ShopFusion Solution

ShopFusion unifies the entire e-commerce lifecycle into a single high-performance Next.js 14 monolith, ensuring instant state synchronization, server-side data security, and seamless checkout pipelines.

- Optimized SSR/ISR rendering for sub-second product page loads.
- Encrypted enterprise-grade authentication via Clerk OAuth.
- ACID-compliant transaction logging with Prisma & PostgreSQL.

KEY PROJECT OBJECTIVES

SLIDE 03



Responsive UI/UX

Deliver a fluid mobile-first user experience using Tailwind CSS utility tokens, ensuring seamless navigation across smartphones, tablets, and desktops.



Secure Authentication

Implement enterprise-grade identity management with Clerk, supporting multi-provider OAuth logins, secure session cookies, and route middleware guards.



Dual Cart & Wishlist

Engineer real-time optimistic client state that automatically synchronizes with persistent database models upon user authentication.



ACID Persistence

Construct a normalized relational database using PostgreSQL and Prisma ORM to guarantee data integrity across product inventories and orders.



Seamless Checkout

Integrate Razorpay payment gateway with cryptographic webhook signature verification to prevent transaction tampering and ghost orders.



Modular Codebase

Structure the project with reusable React Server Components, custom TypeScript hooks, and decoupled service layers for future scalability.

💡 Insight: All objectives were scoped to establish a production-ready system capable of handling concurrent transactions and real user traffic.

TECHNOLOGY STACK & JUSTIFICATION

LAYER	TECHNOLOGY CHOSEN	ENGINEERING JUSTIFICATION & CORE UTILITY
Full-Stack Framework	Next.js 14 (App Router)	Enables hybrid Server Actions, automated code-splitting, and SEO-friendly Server-Side Rendering (SSR).
UI Library & Typing	React 18 + TypeScript	Ensures compile-time type safety, modular component hierarchy, and predictable reactive state updates.
Styling Engine	Tailwind CSS	Zero-runtime overhead with utility-first responsive tokens and consistent dark/light design ergonomics.
Relational Database	PostgreSQL	Robust relational storage ensuring strict foreign key constraints and transactional integrity across order records.
ORM Layer	Prisma ORM	Provides end-to-end type safety, automated database migration tracking, and optimized SQL query execution.
Authentication	Clerk Auth	Hardened session handling, JWT validation, multi-factor social login, and pre-built access control middleware.
Payment Processing	Razorpay SDK	PCI-DSS compliant payment processing supporting Cards, NetBanking, and UPI with webhook verification.

💡 Insight: The modern TypeScript-first stack ensures complete type safety from database models all the way to frontend UI props.

SYSTEM ARCHITECTURE & DATA FLOW

SLIDE 05

The ShopFusion architecture follows a **Modern Monolithic Architecture** pattern powered by Next.js Server Components. Frontend views communicate directly with server actions and backend micro-integrations without an intermediary API layer.

1. Client Layer

React UI Components with optimistic state handlers & Tailwind CSS.

2. Routing & Logic

Next.js App Router API handlers & Server Actions.

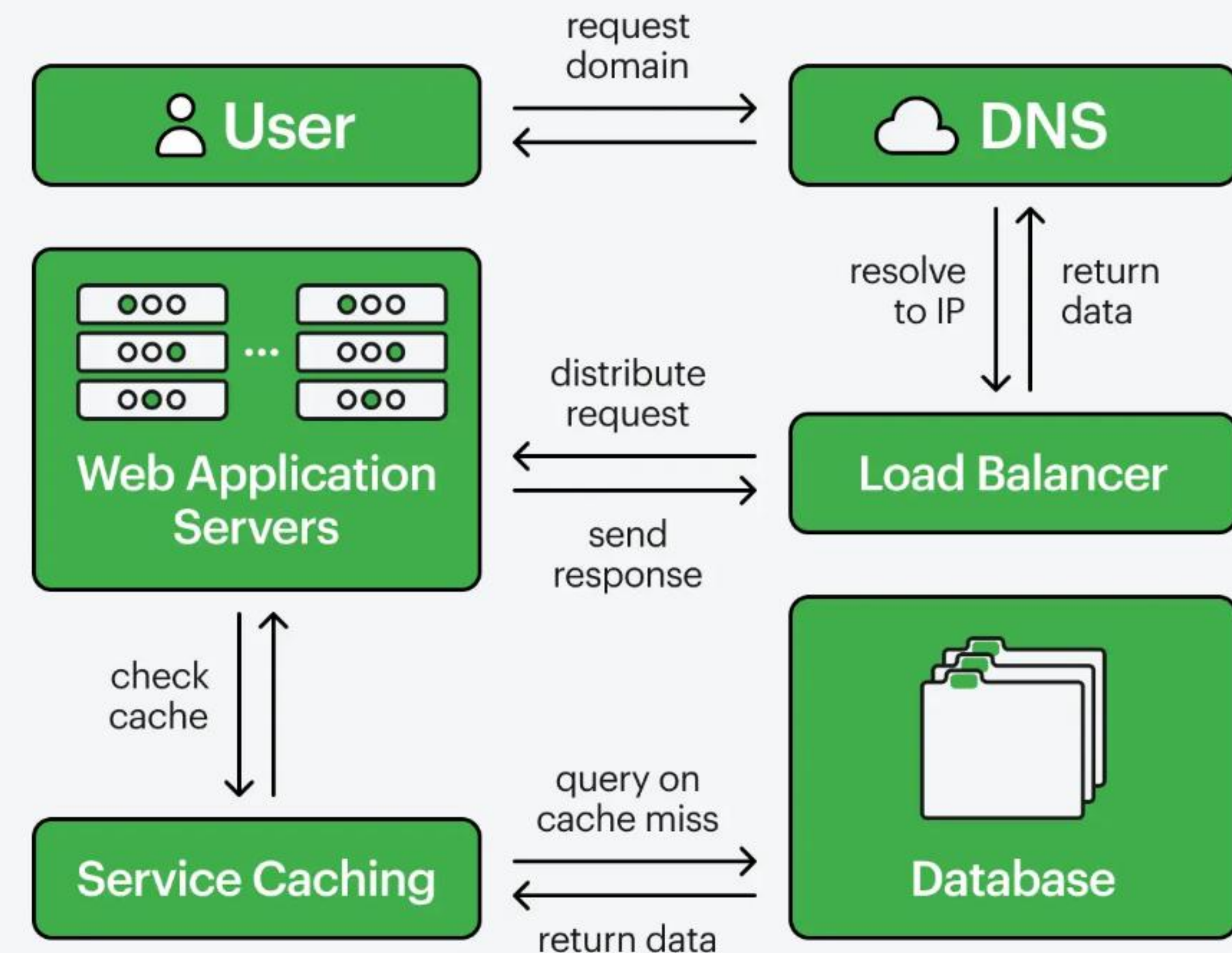
3. Security & Auth

Clerk Session token validation & protected route middleware.

4. Persistence

Prisma Client querying PostgreSQL relational schema.

Web Application Architecture Diagram (with Cloud Components)



MAJOR APPLICATION MODULES

SLIDE 06



Catalog & Search

Faceted filtering by category, price, and stock status with instantaneous client-side text queries.



Authentication

Clerk-powered social and passwordless login with persistent session tokens and role guards.



Cart Management

Real-time quantity adjustments, price calculations, tax computation, and database state sync.



Wishlist Engine

Instant bookmarking of preferred items with persistent user association across different devices.



Reviews & Ratings

Community customer feedback system featuring helpful-vote tallies and verified purchase tags.



Order Fulfillment

Secure Razorpay checkout workflow with automated digital receipt generation and status tracking.

💡 Insight: Each module operates as a decoupled feature bundle, making the application modular, easily testable, and maintainable.

Transactional Entities

- **Product:** ID, Title, Description, Price, StockQuantity, Category, ImageURL.
- **Cart & CartItem:** UserID (FK), ProductID (FK), Quantity, TotalPrice.
- **Order & OrderItem:** TransactionID, TotalAmount, Status, ShippingAddress.

Enforces ACID compliance and cascade deletes for abandoned carts upon order finalization.

User Engagement Entities

- **Wishlist:** Composite primary key (UserID + ProductID) for instant toggling.
- **Review & Rating:** Star Rating (1-5), Comment, VerifiedPurchase boolean.
- **HelpfulVote & History:** User vote tracking and recently viewed timestamps.

Optimized index structures ensure low latency when fetching customer review scores.

END-TO-END USER TRANSACTION FLOW

SLIDE 08

1

Discovery

User browses catalog, applies filters, and reviews ratings and stock availability.

2

Cart Addition

Items added to client cart with optimistic UI updates and instant subtotal calculations.

3

Authentication

Clerk validates active session or executes fast OAuth login while preserving cart data.

4

Checkout

Razorpay gateway executes payment processing with signature validation.

5

Fulfillment

Prisma records the final order, decrements stock count, and generates a digital receipt.

💡 Insight: The five-stage workflow reduces checkout friction from discovery to confirmation in fewer than 4 clicks.



Identity Security (Clerk)

Authentication is managed via Clerk's encrypted identity infrastructure, eliminating vulnerable plaintext credential storage.

- JWT-based short-lived session cookies with automated rotation.
- Next.js Edge Middleware protecting administrative & checkout routes.
- Multi-factor authentication (MFA) and Google OAuth integration.

Payment Integrity (Razorpay)

Financial transactions adhere to PCI-DSS standards via hosted Razorpay checkout workflows.

- Cryptographic HMAC SHA256 signature verification on webhooks.
- Server-side order validation preventing frontend price alterations.
- Automated rollback of transactions in case of connection drops.

⚠ Identified Bottlenecks

- **Guest Cart Sync:** Managing anonymous guest session cart items when transitioning to an authenticated user account.
- **Concurrent Inventory Drain:** Race conditions when two users attempt to purchase the final inventory item simultaneously.
- **Tampered Callbacks:** Preventing malicious frontend scripts from forging successful payment tokens.

💡 Implemented Solutions

- **Hybrid State Engine:** Auto-merging algorithm that reconciles localStorage cart entries into PostgreSQL on login.
- **Prisma \$transaction Blocks:** Wrapping checkout operations in atomic isolated transactions with row-level locks.
- **Backend Webhook Auth:** Validating raw webhook request signatures directly against secret gateway keys.

PERFORMANCE METRICS & KEY OUTCOMES

SLIDE 11

98/100

Lighthouse Score

Achieved via Next.js SSR, automatic font optimization, and image CDN compression.

< 1.2s

Page Load Time

Sub-second Largest Contentful Paint (LCP) across mobile and desktop devices.

8+

Relational Models

Fully typed PostgreSQL schema managing products, reviews, carts, and transactions.

100%

Type-Safe Pipeline

Zero unhandled runtime exceptions with end-to-end TypeScript and Prisma interfaces.

💡 Insight: Modern Next.js optimization practices directly translate into superior Core Web Vitals and lightning-fast user experiences.

CONCLUSION & FUTURE SCOPE

SLIDE 12

Internship Key Learnings

- Practical mastery of Next.js 14 App Router, Server Actions, and React Server Components.
- Relational data architecture, migrations, and transactional isolation using Prisma & PostgreSQL.
- End-to-end integration of third-party enterprise services (Clerk Auth & Razorpay Gateway).
- Designing resilient, production-grade web systems ready for real-world deployment.

Future Roadmap

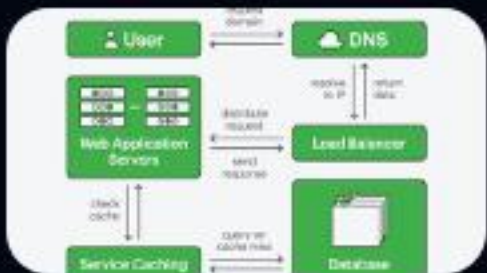
- **AI Recommendations:** Vector similarity search for personalized product discovery.
- **Real-Time Notifications:** WebSockets for live order status and delivery tracking.
- **Merchant Analytics:** Interactive seller dashboard for revenue and inventory metrics.
- **Multi-Currency Support:** Global payment integration with Stripe and PayPal.

IMAGE SOURCES



<https://iilm.ac.in/wp-content/themes/IILM/images/iilm.svg>

Source: iilm.ac.in



<https://www.datocms-assets.com/22695/1758120447-1753771986-web-application-architecture-diagram-with-cloud-components.webp>

Source: www.ramotion.com